Smart Academic Workload System: Core Logic Document

This document provides a comprehensive overview of the main business logic that powers the Smart Academic Workload & Exam Preparation System. The core software components are highly concentrated in the backend structure, mainly bridging the Database elements with computational algorithms formulated in the Flask server.

1. Application Routing & Authentication (Flask)

The system relies on web routes (endpoints) to handle incoming requests and render appropriate HTML views dynamically. Security is prioritized by verifying user session presence.
If a user is not authenticated ("user_id" not in session), they are redirected directly to the login page.

```
@app.route("/dashboard")
def dashboard():
    # 1. Authentication Authorization Check
    if "user_id" not in session: return redirect(url_for("login"))
    user_id = session["user_id"]

    # 2. Extract specific user data
    tasks_df = db.get_tasks(user_id)
    pending_tasks = tasks_df[tasks_df["status"] == "Pending"]
    completed_tasks = tasks_df[tasks_df["status"] == "Completed"]
    subjects_df = db.get_subjects(user_id)

    avg_readiness = subjects_df["preparation_progress"].mean() if not
subjects_df.empty else 0.0

    # 3. Render Dashboard with injected user parameters
    return render_template("dashboard.html",
        username=session["username"],
        total_subjects=len(subjects_df),
        total_pending=len(pending_tasks),
        total_completed=len(completed_tasks),
        avg_readiness=round(avg_readiness, 1)
    )
```

2. Dynamic Stress Score Algorithm

The analytical core of the platform is predicting student workload saturation. The system computes a dynamic Stress Score (0 to 100) utilizing three mathematical penalties based on task quantity, task urgency, and poor exam preparedness.

```
stress_score = 0
num_pending = len(pending_df)

# Factor A: Base Volume Penalty (5 points per pending task, maximum of 40
points)
stress_score += min(num_pending * 5, 40)

if num_pending > 0:
    pending_df['deadline'] = pd.to_datetime(pending_df['deadline']).dt.date
    today = date.today()

    # Factor B: Urgency Deadline Penalty (10 points per urgent task <= 3
days away, maximum of 30)
    urgent_tasks = pending_df[pending_df['deadline'] <= (today +
pd.Timedelta(days=3))].shape[0]
    stress_score += min(urgent_tasks * 10, 30)

if not subjects_df.empty:
    avg_prep = subjects_df['preparation_progress'].mean()

    # Factor C: Exam Anxiety Penalty (Distance from 50% readiness score,
maximum of 30)
    if avg_prep < 50:
        stress_score += min((50 - avg_prep) * 0.6, 30)

# Final Constraint
stress_score = min(stress_score, 100)

# Outcome label mapping
stress_label = "Low" if stress_score < 30 else "Moderate" if stress_score <
70 else "High"
```

3. The Smart Reminders AI Engine

To manage attention efficiently, the system scans through all pending tasks and subjects. It emits context-sensitive JSON objects displaying custom HTML alert messages according to urgency thresholds.

```
reminders = []

# Section 1: Processing Task Deadlines
for _, row in urgent.iterrows():
    days_left = (row['deadline'] - today).days

    if days_left < 0:
        reminders.append({"type": "danger", "message": f"Overdue:
'{row['title']}' was due {-days_left} days ago! Please submit it or mark it
complete."})
    elif days_left == 0:
        reminders.append({"type": "danger", "message": f"Urgent:
'{row['title']}' is due TODAY!"})
    else:
        reminders.append({"type": "warning", "message": f"Reminder:
'{row['title']}' is due in {days_left} days."})
```

```
# Section 2: Processing Exam Objectives Thresholds
for _, row in subjects_df.iterrows():

    if row['preparation_progress'] < row['target_score'] - 20:
        reminders.append({"type": "info", "message": f"Exam Prep: You are
far from your target in {row['name']}. Dedicate some study time soon!"})

    elif row['preparation_progress'] < 50:
        reminders.append({"type": "warning", "message": f"Exam Prep: Your
readiness for {row['name']} is below 50%."})

import random
random.shuffle(reminders) # Return random subset for UI display
return jsonify({"reminders": reminders[:3]})
```

4. Workload Distribution

The system quantifies effort horizontally by mapping projected work volume (estimated_hours) across various academic modules utilizing powerful pandas DataFrame aggregation.

```
# Workload Aggregation Map
workload =
pending_df.groupby('subject')['estimated_hours'].sum().reset_index()

# Returning clean lists directly readable by the Chart.js frontend
workload_data = {
    "labels": workload['subject'].tolist(),
    "data": workload['estimated_hours'].tolist()
}
```

5. Database Architecture & Hashing Security

The database sits precisely behind a functional controller using SQLite. Direct password exposure is strictly prohibited by hashing strategies utilizing generate_password_hash from the Werkzeug Security suite.

```
def verify_user(username, password):
    conn = sqlite3.connect(DB_NAME)
    c = conn.cursor()

    # 1. Look up user by unique username constraint
    c.execute("SELECT id, password FROM users WHERE username=?",
(username,))
    user = c.fetchone()
    conn.close()

    # 2. Verify inputted plaintext hash against the DB
    if user and check_password_hash(user[1], password):
         return user[0] # Returns the integer Primary Key User ID
```

```
return None
```